

Towards Formal Verification of LLM-Generated Code from Natural Language Prompts

ANONYMOUS AUTHOR(S)

In the past few years LLMs have emerged as a tool that can aid programmers by taking natural language descriptions and generating code based on it. However, the reliability of LLM code generation and current validation techniques for it are far from strong enough to be used for mission-critical or safety-critical applications. In this work we explore ways to offer formal guarantees of correctness to LLM generated code; such guarantees could improve the quality of general AI Code Assistants and support their use for critical applications. To address this challenge we propose to incorporate a Formal Query Language that can represent a user's intent in a formally defined but natural language-like manner that a user can confirm matches their intent. We then have a formal specification of the user intent which we can use to verify that LLM-generated code matches the user's intent. We implement these ideas in our system, Astrogator, for the Ansible programming language, widely used for system administration, including for critical systems. The system includes an intuitive formal query language, a calculus for representing the behavior of Ansible programs, and a symbolic interpreter and a unification algorithm which together are used for the verification. A key innovation in Astrogator is the use of a Knowledge Base to capture system-specific implementation dependencies that greatly reduce the need for system knowledge in expressing formal queries. On a benchmark suite of 21 code-generation tasks, our verifier is able to verify correct code in 83% of cases and identify incorrect code in 92%.

CCS Concepts: • **Theory of computation** → **Program verification**; • **Software and its engineering** → **Automatic programming**; **Domain specific languages**.

Additional Key Words and Phrases: Natural Language Programming, Large Language Models, Code Generation, Correctness

1 Introduction

In recent years, Large Language Models (LLMs) have proven to be surprisingly successful at generating code from natural language prompts [20, 31, 40]. The apparent success of LLMs at this task has led some to speculate that prompting LLMs to generate code will fully replace traditional programming, but programming in natural language has many challenges [12] and using LLMs for this purpose has even more [43]. For example, LLMs are known to suffer from hallucinations in code generation, including generating code that uses non-existent APIs and libraries [44] and LLMs have also been shown to struggle with many practical programming tasks that rely on complex functions [33, 45] and involve reasoning about large codebases [23]. Even when the generated code is valid, studies on AI Code Assistants suggest that users have a hard time checking LLM generated code [28], that bugs in LLM generated code are often subtle [13], and the use of AI Coding Assistants can be associated with less secure code [36]. For mission-critical and safety-critical (together “critical”) software, such as software in network stacks, distributed systems protocols, financial software, and embedded systems controllers for critical systems, the use of LLM code generation can pose serious risks as errors can have disastrous impacts. The literature on AI Code Assistants suggests they offer potential benefits to programmers [7, 28, 46] and so the appeal of using them for critical code is apparent, especially as critical software is often difficult and complex. However, because of the significant risks that errors can pose, any code in these systems needs to be correct and significant efforts in formal verification have been focused on such systems [1, 25, 27]. As such, it is essential to provide formal guarantees about any LLM generated code for such systems.

While using tests has been explored as a way to check LLM-generated programs [10, 11] this approach is not sufficient in this context because tests can only guarantee the correctness of the program on the finite set of tested inputs, meaning test cases can *never* completely guarantee

correctness. In critical systems, a guarantee of correctness only a subset of possible inputs is not sufficient because it means failure may be possible. Additionally, the assurance of correctness that tests can offer is highly dependent on their quality [9, 14], including factors like how well they cover the space of possible inputs (including invalid or unexpected inputs) and coverage of the code; because of the many factors that must be considered, developing good tests is difficult [6]. These challenges are exacerbated in settings like system configuration tasks, which can be critical software if used to configure such systems, because identifying how to test something can be very difficult; as an example, how do we test that the Python package `numpy` is installed? It might seem that running the Python command `import numpy` is sufficient but this will succeed if there is simply a `numpy.py` file in the current directory. For these reasons, tests are not sufficient and not a replacement for formal guarantees and formal verification of critical software.

Our broad, long-term, goal then is to provide a formal guarantee that code generated by an LLM from a natural language prompt has the user’s desired functionality. This is a very hard problem that poses several significant challenges. Perhaps the most fundamental is that there is no formal specification: the natural language prompt is inherently informal and may be ambiguous, vague, or incomplete. This means traditional approaches that provide formal guarantees, such as program synthesis and verification, are not immediately applicable: without a formal specification, what property would one prove? Second, even if a formal specification were available, proving the correctness of LLM-generated code with respect to that specification poses additional challenges. One approach to *prove* correctness against a formal specification is to use symbolic execution. This requires the formal semantics of both the target and formal specification languages. It also requires defining a symbolic execution engine and translating the semantics of both languages to the input language of the execution engine. Finally, it requires defining a verification procedure that can prove equivalence between the two executions, a task that is often difficult to make scalable.

In this work, we propose a general solution to the the first problem: of how to obtain a reliable formal semantics for the user’s intent. We also address the other challenges in the somewhat limited setting of a class of Domain Specific Languages. These other challenges build on a long history of work in formal verification and we leave it to future work to apply these techniques to broader classes of languages.

To address the first problem, we propose to add a high-level **formal query language** which is designed to be close to the syntax of natural language but is precisely and formally defined. We aim to design this formal query language to be at a similar level of abstraction to that of the user’s natural language description — using similar concepts and abstractions — and to make it readable and intuitive, so that a user can easily understand the formal query and ensure it reflects their intent. This *user-validated formal query* then gives us a *reliable formal specification of the user’s intent* we can use to verify the LLM-generated programs. While verification remains a very difficult problem, this work offers a bridge between the user’s intent and the realm of formal verification.

As a first step on the rest of the challenges, we focus on Domain-Specific Languages (DSLs), which LLMs often struggle to generate [19]. We believe that providing formal guarantees of correctness may be easier for DSLs because we can use specialized domain knowledge not available in general-purpose programming languages and, since many DSLs are not Turing Complete, some techniques from program verification may be more tractable. In this work, we focus on the Ansible programming language, which is used for IT automation, where correctness can be critical when used to configure critical systems. Ansible has a relatively large user-base and open source community and was acquired by Red Hat in 2015. We also demonstrate, through examples, that our techniques to offer guarantees for LLM generated code can be used for other languages, specifically Bash and a microcontroller programming language (see Section 7.2) but we leave a full development of verification systems for these languages to future work.

```

99 1 ---
100 2 - name: Install and start apache on servers
101 3   hosts: servers
102 4   become_user: root
103 5   become: true
104 6   tasks:
105 7     - name: Install apache server
106 8       ansible.builtin.package:
107 9         name: "{{ 'apache2' if ansible_os_family
108 10           == 'Debian' else 'httpd' }}"
109 11       state: present
110 12     - name: Start apache service
111 13       ansible.builtin.service:
112 14         name: "{{ 'apache2' if ansible_os_family
113 15           == 'Debian' else 'httpd' }}"
114 16       state: started

```

Fig. 1. An example Ansible playbook composed of a single play which configures a set servers by installing and starting Apache server.

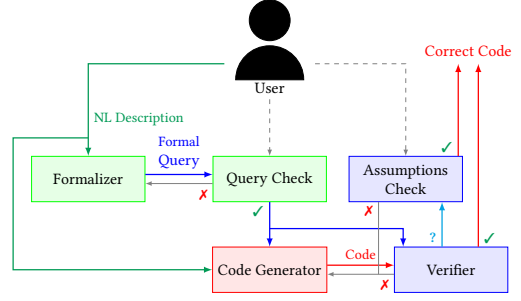


Fig. 2. Overview diagram of our code generation system. The dashed lines between the user and Query Check and Assumptions Check indicate the user needing to review and approve or disapprove.

Our key contributions in this work are as follows:

- (1) Formalize our notion of programming using natural language and guaranteeing the code’s correctness (Section 3).
- (2) Propose the idea of an (artificially introduced) formal query language that captures the user’s intent in precise but high-level terms that can be reviewed by the user (Section 4.1).
- (3) Present Astrogator, an implementation of such a formal query language and verifier for the Ansible programming language (Section 5).
- (4) Evaluate Astrogator on a benchmark suite of 21 LLM code-generation tasks for Ansible programs from natural language descriptions, showing that it is able to formally prove correctness of generated code in 83% of cases, and to precisely identify incorrect LLM generated code in 92% of cases. We also detail the failures of Astrogator and show that they are due to minor limitations in our implementation and testing set-up, and not due to limitations of the overall approach (Section 6).

The other sections of this paper discuss background on Ansible and errors in LLM-generated Ansible code (Section 2), the generalizability of our approach (Section 7), comparisons with related work (Section 8), and our conclusions (Section 9).

2 Background

2.1 Ansible

Ansible is an open-source system used to automate many IT tasks involved in the management of remote systems (such as servers or virtual machines) including provisioning, system configuration, and application deployment and orchestration [4, 38]. Ansible runs on one system, known as the *controller*, connects to the remote systems being managed via SSH, and sends commands from the controller to the remote systems and information and responses back. This simple architecture enables Ansible to be used for a wide variety of remote systems.

Ansible programs, called *playbooks*, are written in YAML [3]; an example is shown in Figure 1. A playbook is composed of a list of *plays* which each specify configuration to perform on a particular set of remote systems; our example contains a single play. Each play contains configuration information and a list of tasks to be performed in-order. In our example, the configuration (lines 2 through 5) includes the name field, which provides a natural language description of the play; the hosts to execute the play on (here “servers”, the machine host names this refers to would be defined

elsewhere); and the user to execute the play as on the remote machine (become: true causes execution as the user specified by become_user). Finally, *tasks* are composed of configuration options and a *module* which specifies the behavior of the task. Task configuration can also include a name field, a when clause that causes the task to only execute if a condition is true, a loop clause that executes it on a list of elements, and so on. Ansible modules are like library functions that define a particular set of operations that can be performed; they are distributed as *collections* such as the built-in collection `ansible.builtin`. Our first task (lines 7 to 11) invokes the builtin-in package module which is used to manage system packages; the arguments to this module include name, which specifies the packages, and state, which specifies whether to install or uninstall them. In this task the name argument is a Jinja expression, that is evaluated at execution time to `apache2` if `ansible_os_family == 'Debian'` (the system is Debian based) and otherwise to `httpd`; this allows us to specify the correct package name based on the OS. The other task (lines 12 to 16) uses the service module to start the Apache server service. Executing the playbook executes both tasks and so will install and then start Apache server and works on both Debian and RHEL.

2.2 Errors in LLM-Generated Ansible

While a detailed study of errors in LLM generated Ansible is beyond the scope of this work we briefly detail two subtle but important errors we observed regularly. The first are incorrect package installations. For example to install the Python package `numpy` on Red Hat Enterprise Linux (RHEL) we can install the `pip` package `numpy` or the `dnf` packages `numpy`, `python-numpy`, or `python3-numpy` but on Debian we must install the `apt` package `python3-numpy`. We have seen LLM-generated code that uses `pip` on all systems, an incorrect package name on Debian, or `apt` on all systems; in all these cases the program fails to install `numpy` in some cases. Errors like this are subtle as they require knowledge of Ansible, to understand the generated code, but also the underlying operating systems and package managers, to know the package names and package managers that can be used. Our other example comes from permanently setting an environment variable; this is done by adding a line of the form `key=value` to the `/etc/environment` file. Ansible has a module specifically for adding and replacing lines in files; to replace a line you specify a regular expression to search for and if a line matches it will be replaced. In some generated code we observe a bug where it uses a regular expression like `^key` which will match and replace any line with a variable which begins with `key`, such as `keys`, and therefore this error can result in unintentionally replacing other environment variables. These observations offer guidance on the kinds of errors we need to detect to ensure correctness of LLM-generated Ansible.

3 Problem Statement

We are interested in providing correctness guarantees for code generated from a natural language description; to do this we need to first understand what it means for a program to be *correct*. Informally, what we mean is that the program acts as the user intends it to, but we cannot provide any formal guarantee of this if the user's intention is only in their mind and expressed, likely imperfectly, in natural language. As a concrete example, consider a natural description like "create a file at `/some/path`", this is missing significant details like the contents, owner, group, permissions, and so on. These missing details are important: the user likely has a vision that includes some of these details (for instance, they may want an empty file) and the system they want to configure may specify others (such as what users exist to own it).

To formalize this problem, let our target language be $\mathcal{T}$ (Ansible in our case). The user has some *intent* they want a program in $\mathcal{T}$ to satisfy; this is likely not a single program as there are generally many ways to achieve the same thing and there may be details the user does not care about, for instance if they want to clone a git repository, if git is already installed it is not necessary to install

it but it does not hurt either. Therefore, we can describe the user’s intent as some set $\mathcal{U} \subset \mathcal{T}$; note importantly that we have *no way* of directly testing membership in this set (the only option is to ask the user but this risks mistakes slipping through). With this, though, we can now formalize our informal notion of correctness: a program $p \in \mathcal{T}$ is *correct* if and only if $p \in \mathcal{U}$.

The user then writes some natural language description $\mathcal{d}$ to express their intent. At this point, we still do not have any way to determine whether a program p is correct as the only indication of the user’s intent is $\mathcal{d}$ which lacks formal semantics. To fix this, we need to introduce some formal specifications that allows us to test correctness. For this, suppose that we have some formal specification language $\mathcal{S}$ (in our case the Ansible Formal Query Language discussed in Section 5.1) and some semantics function $\llbracket \cdot \rrbracket : \mathcal{S} \rightarrow \mathcal{P}(\mathcal{T})$, which defines the set of programs that satisfy a particular specification. We also assume that given a program $p \in \mathcal{T}$ and specification $s \in \mathcal{S}$ we can decide whether $p \in \llbracket s \rrbracket$. Finally, suppose that we have some function $\mathcal{F}$ which converts a natural language description into a formal specification.

Because $\mathcal{F}$ takes a natural language description as input there must be instances where its output does not match the user’s intent since the input may be ambiguous and $\mathcal{F}$ can only output one unambiguous formal specification. This is inherent to any attempt to automatically formalize a natural language description. The only solution to this problem is to show the generated specification $s = \mathcal{F}(\mathcal{d})$ to the user and ask them to check that it matches. Once the user approves a specification we believe that the specification s should satisfy $\llbracket s \rrbracket \supseteq \mathcal{U}$ indicating that the specification contains the user’s intent but may contain other programs as the user’s intent may not be fully expressed in their description and thereby the specification.

Now, we can generate candidate programs p until we find one that satisfies the specification, $p \in \llbracket s \rrbracket$. With this, we are close to a correct program but we need $p \in \mathcal{U}$, not just $p \in \llbracket s \rrbracket \supseteq \mathcal{U}$. With the information at hand: the natural language description $\mathcal{d}$ and the formal specification s , there is no way to check this. We expect that s is close to the user’s intent and so if $p \notin \mathcal{U}$ this should be because of how p fills in under-specified details. Therefore, we need to identify how p differs from an *unrefined* version of s , which does not make any particular choices for the under-specified details. To do this, suppose we are able to compute an operation $p \setminus s$ which produces a high-level description of these choices. Exactly what this operation and description look like will vary depending on the target language $\mathcal{T}$, but for Ansible these differences include details like the assumption that a program is installed or a particular group exists or that the program installs an unspecified program or sets a file owner when none was specified. The result should be high-level so that it can be easily understood by the user and they can confirm whether or not the choices match their intent. By ensuring that the program $p \in \llbracket s \rrbracket$ we reduce the space of possible issues that the user must consider, but ultimately the task of determining whether $p \in \mathcal{U}$ is one that only the user can solve.

4 Approach

We have proposed two key components that are the focus of this work: the **Formal Query Language**, our formal specification language $\mathcal{S}$, which expresses the user’s intent in a formal manner, and the **Verifier** which, given a program p and formal query s , determines whether p satisfies s ($p \in \llbracket s \rrbracket$). We envision that, in the future, these ideas could be incorporated into a full end-to-end system, like shown in Figure 2, that would guarantee the correctness of generated code as described in Section 3.

As described earlier, the Formal Query Language must be understandable to the user because they must check that it matches their intention. We discuss the design considerations to satisfy this in Section 4.1. For our “Formalizer” ($\mathcal{F}$) we propose to use an LLM; we note that the risk of mistakes by the LLM is not an additional problem here since there is no formalizer that can be correct all the

time. While we do not address the problem of generating formal queries in this paper, we believe it is suitable for LLMs, even though they are not trained on the formal query language, because in-context learning, where a few examples are provided to an LLM in the prompt, has shown efficacy at enabling LLMs to generate code in custom and domain-specific languages [24]; LLMs have also been used before to formalize natural language descriptions into formal specifications such as postconditions [15] and the generation of the formal query is not too dissimilar to the explanations of their work that LLMs often give.

Once the user is satisfied with a formal query, we can use it and their natural language description to generate programs (the “Code Generator” in Figure 2). Note that the code generated by this step does not need to satisfy the formal query; in this work we actually do not use the formal query in this step and simply provide the natural language description to an LLM though we believe that providing the formal query could clarify the user’s intent if there is ambiguity.

Finally, a generated program is checked against the formal specification by the “Verifier”. We discuss the role of the verifier in detail in Section 4.2. As Figure 2 shows, the user may be asked to check the choices a program makes regarding the under-specification of the query ($p \setminus s$) via the “Assumptions Check”. In this work, we can identify such assumptions (i.e., our verifier calculus $p \setminus s$) but in our evaluation we accept all programs where $p \in \llbracket s \rrbracket$.

4.1 Design Considerations for a Formal Query Language

As described earlier, to provide any formal guarantee of correctness we need a formal specification of what the program should do. We do not need this formal specification to be complete, indeed program synthesis is often done on incomplete specifications, but do need this specification to be unambiguous. As a concrete example, consider the following description for a desired Ansible program: “copy a file from /path/a to /path/b.” In the context of Ansible this could mean duplicating the file on the remote system, copying file /path/a from *the Ansible controller* to /path/b *on the remote system*, or from the remote system to the controller! Any of these interpretations is conceivably correct, i.e., the query is ambiguous. While we intended this example as the first interpretation, we find that all six LLMs we prompted *only* generated code to copy a file from the controller to the remote system, potentially because this is the default mode implemented in the copy module. This example raises a key design question:

(Q1) How do we ensure the formal specification matches the user’s intent?

Section 3 provides our answer: the user must check that the specification is correct. We therefore need a language, that we call a **Formal Query Language**, that is designed to formalize the user’s intent described in natural language. Since the user must check the specification in this language, which we call a **Formal Query**, we want to make it easy and intuitive for the user and so the formal query should be close in syntax to natural language so that the user can read and understand it. This means that our formal specification must be high-level and appear natural language-like. The idea of natural language-like formal languages is not new, Controlled Natural Languages [26] have been explored before and it has been shown that such languages can be easy to understand [21] suggesting that can ensure that the specification is correct. A further design question arises, though:

(Q2) How do we ensure the user does not incorrectly believe a query matches their intent?

For example, as described previously, on Debian the package name for numpy is python3-numpy, but a user may not know this off the top of their head and might believe a reasonable name like numpy is correct. Our solution is that the formal query language must preserve certain high-level concepts, so rather than using package names we use *package descriptions* such as “numpy” or “apache server” and use a trustworthy knowledge base to determine the correct package names and similar facts. This use of high-level concepts and a knowledge base is part of why we are focusing

currently on Domain-Specific Languages; the domain provides the high-level concepts that are needed and provides the information needed to construct the knowledge base. We discuss our use of these design principles to build a formal query language in Section 5.1.

4.2 Program Verifier

Along with the Formal Query Language, the Program Verifier is the other key component to our approach. The Verifiers's role is to take a generated program and a formal query and determine whether or not the program matches the *user's intent*. This is a significant challenge as there is no way to do it directly, we can only use the formal query and seek feedback from the user. Let us consider the earlier example of cloning a git repository to exemplify this challenge. In our experiments we find that some LLM generated programs for this will simply invoke ansible's git module (which requires git to be installed) and others will first install git before using it. Both of these are reasonable: if the user's intent is to run the program on a machine with git installed they would likely be fine with either version (though there may be circumstances where they prefer or need the former) but if the user's target does not have git installed they need the second version. A convenient solution for the Verifier would be to require that the formal query include a full description of such details, so specifying to install git if needed, but this would violate design question (Q2) as the user would need to think of these details and failing to do so, here simply failing to note whether they need to install git, could result in accepting incorrect code.

Thus, as described in Section 3, the Verifier's role is two-fold, to ensure that the program generally matches the query ($p \in \llbracket s \rrbracket$) and therefore to detect certain errors like installing the wrong package or failing to create a file, *and* the Verifier must also identify how the program fills in under-specified parts of the query ($p \setminus s$) and present them to the user who must check them. The details of how a verifier does these things will depend highly on the target language and formal query language; we discuss our approach in Section 5.2.

5 Formal Query Language and Verification of LLM-Generated Code for Ansible

We have implemented this design for Ansible in our system called Astrogator. In Astrogator we have developed a formal query language and program verifier which together can be used (as shown in our Evaluation in Section 6) to reliably identify correct and incorrect LLM generated Ansible code. We describe the design of our Formal Query Language in Section 5.1. We then discuss the program verifier, which is built on a novel calculus we call the **State Calculus** that we use to define the semantics of Ansible (Section 5.2.1), a symbolic interpreter for this calculus (Section 5.2.2), and a unification process that matches the expected and actual behaviors of a program (Section 5.2.3). Finally, we explain how formal queries and Ansible programs are compiled into this calculus (Sections 5.3 and 5.4) and describe the formal query language's knowledge base (Section 5.5).

5.1 Ansible Formal Query Language Design

As discussed already the role of the Formal Query Language is to provide a formal expression of the user's intent and must do so in a way so as to be understood, and reviewed, by the user. We identified two key questions to consider in designing our Formal Query Language; our answers are:

- (A1) The formal query language must be sufficiently high-level and interpretable that a user can easily understand whether a query matches their intent or not.
- (A2) Determining whether a query matches the user's intent must not rely on specialized knowledge (such as package names or assumptions made by Ansible modules).

These principles ensure that the user can verify a given query is correct and that errors do not go undetected by the user.

```

query ::=  $\epsilon$  | sentnc | sentnc . query
sentnc ::= atom | atom ; sentnc | atom ; and atom | if cond then sentnc otherwise sentnc
atom ::= verb desc args
cond ::= cond or cond | cond and cond | os is Debian | os is RedHat | desc args exists | ...
verb ::= create | delete | copy | move | enable | disable | install | ...
desc ::=  $\epsilon$  | file | bashrc file | directory | user | group | numpy | apache server | ...
args ::= sep binds | sep val
sep ::= to | from | at | with | ...
binds ::= id = val [, binds]
val ::= (id | string | int)*

```

Fig. 3. Grammar of the Ansible Formal Query Language.

Natural Language Description	Formal Language Query
Move the file at /scratch/file.txt to /home/user/data.txt	move file from remote "/scratch/file.txt" to remote "/home/user/data.txt"
Copy all .war files from /tmp on the remote machine into /backup on the controller	copy files with glob="*.war" from remote /tmp to controller /backup
Create a Python 3.10 virtual environment at /user/home-venv and install numpy in it	create virtual environment with python="3.10" in /user/home/venv; install numpy in virtual environment at /user/home/venv
Install zsh, set it as the user 'dev's default shell, and ensure its configuration file exists	install zsh; set default shell for user="dev" to zsh; if zsh configuration file for user="dev" not exists then create zsh configuration file for user="dev"
Enable passwordless sudo for a 'wheel' group and create a 'ansible' user in that group	enable passwordless sudo for group=wheel; create user with name=ansible in supplemental groups=wheel

Table 1. Example Natural Language descriptions and Formal Language Queries for Ansible programming tasks. Colors and fonts of the formal queries correspond to the non-terminals in Figure 3.

From these principles we have developed a Formal Query Language for Ansible; a fragment of the grammar is shown in Figure 3 and a number of examples are shown in Table 1. Towards principle (A1), queries are designed to mimic the structure of natural language; they are sequences of sentences which are, in turn, sequences of basic atomic queries or a conditional sentence like the fourth example, where if zsh configuration file for user="dev" not exists allows the query to specify what should happen in different circumstances. Atomic queries specify simple imperative actions and are defined by a verb (such as move in the first example or copy in the second), a description (like the package description zsh or file description like zsh configuration file in the fourth query), and finally additional arguments defined either by a separator and a value (such as at /user/home/venv in the third query) or a separator followed by key-value argument pairs (like with glob="*.war" in the second example). Separators act both to separate the description from the arguments and when followed by a value act as the variable name, so to remote "/home/user/data.txt" in the first example sets the to argument (specifying the destination) to the given path.

In following the (A2) principle, our design reduces the effort required by users to verify a query by preserving high-level descriptions, such as numpy in the third example (rather than using the actual package name), zsh configuration file for user="dev" in the fourth (rather than using the actual path), and passwordless sudo in the fifth (rather than describing the changes needed to the sudoers file). Details on how we achieve this are discussed in Section 5.5.

Returning to the example Ansible program in Figure 1, which installs and starts Apache server on Debian and RHEL, a formal query for this problem is: if os is Debian or os is RedHat then install apache server; and start apache server service. This example illustrates that our formal queries are significantly easier to understand than the Ansible code, are close to natural language in structure while still being precise, and are easier to verify the correctness of.

5.2 State Calculus and Interpreter

In order to verify the correctness of Ansible programs we compare the behavior of the program and expected behavior defined by the formal query. To do this, we have developed the **State Calculus** which provides a clean mechanism to describe the semantics of Ansible programs (Section 5.2.1). Given an Ansible program and a formal query, both compiled into the State Calculus (see Sections 5.3 and 5.4 for details), we make use of a symbolic interpreter to identify their behaviors (Section 5.2.2) which we compare to determine whether the program satisfies the query (Section 5.2.3).

5.2.1 State Calculus. Programs in languages like Ansible and Bash are executed mostly for side-effects in the form of changes to the system state. We can model such programs as executing with some current system state they read information from (such as whether a file exists) and make modification to. This gives rise to our **State Calculus**, which is a simple calculus designed to model programs that operate in this manner.

States. We define states generically, so as to be applicable to Ansible, other languages, and even other domains. States, denoted by σ , are defined in Figure 4(a); formally, they are a pair of functions, but we can think of it as a set of two kinds of values: attributes and elements. Attributes, described by *attribute labels* from some set $\mathcal{A}$, map to a value v ; these attribute values are used to describe information in the state that has a value. For example, we might have an attribute `os` whose value describes the operating system we are running on or an attribute `file_mode` describing the file permissions of a certain file. Elements are used to model the existence or non-existence of certain objects in the state, such as files or users. An element is specified as an *element label* from $\mathcal{E}$ and a value v ; for instance, if `file` $\in \mathcal{E}$ then `file(/etc/passwd)` is an element referring to the `/etc/passwd` file. Where attribute labels identify a piece of information, an element label identifies a class of objects and when applied to a value it identifies a particular object. In our states, elements map to either the special value $\bullet$, indicating the element does not exist, or to a *nested state* σ , which allows us to apply attributes on an element. For example, the `file(/etc/passwd)` element can map to a state with attributes describing the owner, the file permissions, and so on. As an example, a state with `file(/etc/passwd)  $\mapsto$  [owner  $\mapsto$  root, mode  $\mapsto$  0644]`, `file(/root/notes.txt)  $\mapsto$   $\bullet$` indicates that the `/etc/passwd` file exists, is owned by root, and has permissions 0644, and the file `/root/notes.txt` does not exist.

Syntax. The grammar of the State Calculus is shown in Figure 4(c), much of which is fairly standard. A program p is a sequence of statements s which are a mix of standard and unique forms. These begin with variable assignment $x \leftarrow \dots$ where the right hand side is a pure expression e (here pure means they do not read or modify the state), or special forms $\alpha(e)$ or **get** a . $\alpha(e)$ represents the invocation of a *stateful function* α . For Ansible, these stateful functions include the Ansible modules but can be any function that can be defined by a program in the State Calculus. The other unique form, **get** a , is used to read the value of the attribute a ; which is either an attribute label from $\mathcal{A}$ or an attribute label on an element $t.\mathcal{A}$, where the element t can be a single element, $\mathcal{E}(e)$, or a sequence of nested elements. For example, we load the owner of a file with $x \leftarrow$ **get** `file(/path).owner`. The next statement form, **add** c , is used to modify the state by assigning a value to an attribute, $a \leftarrow e$, or adding an element l . As an example, the statement

442		
443	$\sigma \equiv (\mathcal{A} \rightarrow \nu) \times (\mathcal{E} \times \nu \rightarrow \tau)$	$p ::= s^*$
444	$\tau ::= \sigma \mid \bullet$	$s ::= x \leftarrow e \mid x \leftarrow \alpha(e) \mid x \leftarrow \text{get } a \mid \text{add } c \mid \text{ret } e \mid \text{fail}$
445	$\mathcal{E}$: the set of element labels	$\mid \text{if } e \text{ then } p \text{ else } p \mid \text{exists } l \text{ then } p \text{ else } p$
446	$\mathcal{A}$: the set of attribute labels	$\mid \text{case } e \text{ of } L(x) \Rightarrow p \mid R(x) \Rightarrow p$
447	ν : the set of values	$\mid \text{foreach } x \text{ in } e \text{ do } p$
448		$e ::= x \mid \ell \mid \langle e, e \rangle \mid \rho(e) \quad a ::= \mathcal{A} \mid t.\mathcal{A} \quad c ::= a \Leftarrow e \mid l$
449	(a) Formal Definition of States	$l ::= t \mid \overline{\mathcal{E}(e)} \mid t.\overline{\mathcal{E}(e)} \quad t ::= \mathcal{E}(e) \mid t.\mathcal{E}(e)$
450		x : variables α : stateful functions ℓ : literals
451	$\mathcal{J} \equiv (\mathcal{A} \rightarrow \nu) \times (\mathcal{E} \times \nu \rightarrow \tau)$	ρ : pure functions
452		
453	(b) Symbolic State Definition	(c) Syntax of the State Calculus

Fig. 4. Description of the State Calculus

454
455
456
457 **add** file(/path).owner $\Leftarrow$ “user” sets the owner of the /path file. We can also specify a *negated*
458 *element*, $\overline{\mathcal{E}(e)}$, to remove an element (i.e., map it to $\bullet$). So **add** file(/path) represents deleting the
459 file. The next two forms, return and failure, are standard.

460 The remaining four statement forms are control flow. if-then-else statements, for-each loops, and
461 case statements are all standard and behave as expected. The fourth is the *exists-then-else* statement
462 which acts like an if-then-else statement but the condition tests whether an element l exists (does
463 not map to $\bullet$). The exists-then-else statement allows us to read elements from the state and serves
464 as a complement to **get** which reads attributes. Finally, the expression forms in the calculus are
465 mostly standard, with variables x , literal values ℓ , pairs $\langle e, e \rangle$, and finally function calls to *pure*
466 *functions* $\rho(e)$, which include built-in operators.

467
468 **5.2.2 Symbolic Interpreter.** To determine the behavior of programs in the State Calculus, we develop
469 a symbolic interpreter that determines the behavior on all possible initial states. The result of
470 the interpreter is a set of *symbolic initial-final symbolic state pairs* that reflect the behavior of
471 the program under different initial states. To do this, the interpreter tracks both an initial state
472 and the cumulative modifications made to the state (which we often imprecisely refer to as the
473 “current state”). These initial and cumulative states are defined as symbolic states $\mathcal{J}$, as defined in
474 Figure 4(b) where the attribute and element maps are partial functions to record only the attributes
475 and elements that are relevant to the program. The design and implementation of the interpreter
476 are mostly straightforward. The main complexity is handling symbolic values and the state which
477 we discuss below.

478 The three constructs dealing with the state are **get**, **add**, and **exists**. Of these, **add** is the simplest,
479 we simply record the addition into the current state. For **get** and **exists** there are two scenarios,
480 either we can determine the result based on the existing state or the result depends on undefined
481 parts of the state. The first scenario is the simplest; for a **get** we search the state for the attribute
482 and find a value for it in the current or initial states and we simply use that value (if we find the
483 attribute in both we use the value in the current state as this is the up-to-date value). Similarly, for
484 **exists**, we find the element bound in the current or initial state (again preferring what we find in
485 the current state) and take the appropriate branch.

486 The other scenario is that the attribute or element being read is undefined. In this case, to
487 interpret the statement, we need to define this missing piece. For **get** all we need to do is add the
488 attribute to the state and give it a symbolic value representing that it can have any value. The one
489 complexity is determining whether to add it to the initial or current state; in general we add it to
490

the initial state (since generally the properties attribute represent depend on the initial system configuration) but if the attribute is nested on an element not present in the initial state we add it to the current state instead. As examples, the first case occurs when a file existed originally and we read the owner which is some (unknown) value while the latter case occurs when a file was created and we read the owner which is some unknown value as it was not explicitly set. Handling **exists** is similar, except we have to handle both cases: where the element exists or not (is bound to **•**). To do this, we will separately update the state for both cases and interpret the **then** and **else** branches respectively. For updating the state we always update the initial state since elements cannot be unspecified like attributes can and so much always originate from the initial state.

The last interesting pieces of the interpreter are how symbolic values are handled by if-then-else, case, and foreach statements. For if-then-else and case statements when the condition or scrutinee is symbolic we must interpret both branches, like we did for **exists**. When we do this we record the value assumed for the symbolic value (i.e., true or false for a condition based on the branch we take) so the value can be reused again later if needed. Handling for-each loops over symbolic values is more challenging. For a concrete list, we simply interpret the body of the loop once for each element of the list, but for a symbolic value we do not know the elements but still want to execute the loop body so that the side-effects of the loop are performed. To do this, we create a new symbolic value to represent the elements of the list and interpret the loop body *once* using this symbolic value as the element. We then record that this symbolic value represents all elements of the list, information which is used in our verifier, as described below. While this approach to handling symbolic lists is imprecise, loops in Ansible are simple enough that this approach captures their general behavior.

5.2.3 Program Verifier. Finally, to verify that the behaviors of the program and query match we compare the results of interpreting them; we call this component the *unifier*.

Basic Operation. By compiling the formal query and Ansible program to the State Calculus (Sections 5.3 and 5.4) we can run the symbolic interpreter to produce a set of symbolic initial-final state pairs for each (we refer to these as the query’s and program’s state pairs, respectively). To verify that the program’s behavior matches the formal query’s we need to ensure that they behave the same under the same initial conditions; the initial conditions are represented by the initial state and the behavior of the program (i.e., what its side-effects are) by the final state. To do this, for each of the formal query’s state pairs s we need to find one of the program’s state pairs q such that q ’s initial state is *consistent* with s ’s and their final states are the same; such a pair indicates that under the initial conditions described in s , the program behaves the same as the query. Note that we only require q ’s initial state to be *consistent* with s ’s, rather than the same; this is because the program is not required to rely on all the assumptions that the query does. For example, a query like `if os is Debian install git` will have an initial state that assumes the OS is Debian, but the program may not ever check the OS since it is not needed (since the package name is always the same) and so that assumption may not appear in the initial states of the program. The final states we do require to be the same, since the program must behave the same as the query, and if anything is missing from the final state it means the program did not behave as expected.

Because the interpreter’s results often involve symbolic values, this process also involves some unification-like techniques where symbolic values may be unified or concretized. As an example, based on a problem in our benchmark suite, suppose we have a query like `if os is Debian then create file "Debian". if os is RedHat then create file "RedHat".`, when we interpret this query these two file names appear directly in the resulting state pairs but a generated program might just use Ansible’s OS family variable as the filename because it has the appropriate value, and so in the program’s resulting state pairs the specific names “Debian” and “RedHat” are not

used, instead a symbolic value is used. This program works, and so to handle this we need to be able to concretize symbolic values in the program's state pairs into concrete values to match the query's. We can also need to unify symbolic values between the query's and the program's state pairs, for instance when both have a file with some unknown contents; note that we will only unify a symbolic value which represent elements of a symbolic list with another value that represents the value of another symbolic list (and that we can unify their symbolic lists), this ensures that we preserve the knowledge that they represent many values at once. Note that we cannot concretize symbolic values in the query's state pairs to match a concrete value in a program's state pair, because a symbolic value in the query's state pair indicates that the behavior is expected for all possible values and so the program should work for all values as well.

Handling under-specified queries. As discussed in Section 4.2 it is often the case that a formal query is an under-specification and this means that a generated program may be valid even if makes additional assumptions (like that a certain program is already installed) or performs additional actions (like install an additional program) compared to the formal query. In addition to allowing this, the unifier also needs to accumulate these assumptions and actions so they can be shown to the user (this is $p \setminus s$ as described in Section 3). To do this, we have to change how we check initial states for consistency and loosen the requirement on final states. For initial states, we already allow programs to have fewer assumptions than the query and now we need to also allow the program to have additional assumptions; thus consistency is now just that if the program's and query's initial states contain a common assumption then it is consistent (i.e., they do not disagree about the existence of an element or the value of an attribute); any assumptions in the program's initial state not present in the query's initial state are reported as additional assumption. Similarly, for final states, we had previously assumed that the program's and query's final states must be the same but now we permit the program's to include additional actions (i.e., additional elements or attributes) and we report any of these as additional actions; importantly it is still the case that all actions in the query's final state must be matched exactly in the program's final state to ensure the program performs the actions expected of it.

5.3 Compiling Formal Queries to the State Calculus

Compiling a formal query to the State Calculus is mostly straightforward; a query is a sequence of atomic queries, the generated code of which are composed together in sequence. Conditional queries are compiled into if-then-else statements after compiling the condition; boolean combinations of conditions are compiled to pure functions that perform these operations, and the base conditions, such as checking the OS or whether a file exists, are compiled into specific code for that purpose such as reading an attribute that represents the OS and checking its value.

Compilation of atomic queries is based on the verb and description. For example, creating a file and a directory generate different code, but creating a file always generates the same code regardless of the description of the file: we add a file element with the appropriate path and set attributes based on the arguments provided. Similarly, installing a package is always the same: we add an element indicating the package's presence and set attributes like the package manager. Other atomic queries are similarly straightforward. Currently the set of verbs we support, and the outline of how to compile them, are built-in to the compiler but we believe these could be made extensible easily, such as through our knowledge base which is already used for handling descriptions (see Section 5.5).

5.4 Compiling Ansible Programs to the State Calculus

To compile Ansible programs into the State Calculus we use a relatively simple compiler to translate playbooks. This compiler relies on definitions of Ansible modules (modules are like library functions that Ansible programs are built from) that provide type information and the semantics as a program in the State Calculus. To facilitate these definitions, we develop a *module definition language*. We describe the Ansible compiler and module description language in turn.

5.4.1 Ansible Playbook to State Calculus Compiler. The actual compilation of an Ansible program into the State Calculus is surprisingly simple. We generate code for each task in each play in sequence and so the core of the procedure is just generating code for an individual task. The module itself becomes an invocation of a stateful function for that module and other features, like conditional execution, storing the returned value, and iterating over a list are straight-forward to handle. The main complexity then is constructing the input as this relies heavily on type information to ensure the arguments used are appropriate and have the correct types. This compilation must also identify built-in Ansible variables, such as those providing information on the OS, and convert them into reading appropriate attributes.

5.4.2 Module Description Language. As mentioned above, the definitions of modules and their types are essential to compiling and interpreting Ansible programs. It would be ideal if this information could be extract from an existing source, such as a module's documentation or (Python) implementation but unfortunately neither is suitable. While some type information could be automatically extract from the documentation important details like when two arguments are mutually exclusive are only described in natural language. As for the implementations, the code is too low-level and complex to extract the core behavior we are looking for. We therefore designed the module description language to facilitate defining the behavior and types of modules. The language includes features for specifying Ansible's module argument, which include optional arguments, some with default values, and sets of arguments where exactly one is required. Many Ansible modules also have arguments that must take a value from some list of values which we represent with custom enum types. The module description language also enables defining new elements and attributes as needed for the modules being specified.

Figure 5 shows a simplified definition for the Ansible copy module, the omitted details deal with the many additional arguments (like file owner and mode) the module supports and details for copying directories. This demonstrates the features we have described. Arguments are defined on lines 2 (dest is a required argument), 3 (exactly one of src and content is required), and 5 (remote_src is optional). When arguments are optional, or one of several options, we can check if a particular one was provided like on line 4 where in the then-branch src was provided and can be used (see line 10) while in the else-branch content was provided and can be used (see line 15). Similarly, we check whether remote_src was provided on line 6. The module's behavior relies a lot on the state, for instance line 9 ensures the source file exists, and lines 10 and 15 set the contents of the destination file based on the source file and contents argument, respectively.

We have designed the module language to make it easy to write specifications for modules, since a specification is needed for every supported module. Towards this goal, it gives significant flexibility in deciding the level of detail tracked in the state. For example, we model file systems at a rather coarse level, like shown in Figure 5, by just tracking the path and system we are referring to, but this approach does not model hard-links, where two files share the same inode, correctly; if we care about such details the module language provides the flexibility to model them, perhaps by defining an inode attribute for files and having an inode element that maps an inode number to


```

638 1 module ansible.builtin.copy -> bool {
639 2     (dest: path)
640 3     (src: path | content: string)
641 4     if provided src {
642 5         [remote_src: bool]
643 6         let src_loc = provided remote_src
644 7             ? (remote_src ? file_system::remote : file_system::controller)
645 8             : file_system::controller;
646 9         if exists file(src, src_loc) {
647 10             file(dest, file_system::remote).content = file(src, src_loc).content;
648 11         } else {
649 12             return false;
650 13         }
651 14     } else {
652 15         file(dest, file_system::remote).content = content;
653 16     }
654 17     return true;
655 18 }

```

Fig. 5. A simplified module description for `ansible.builtin.copy`

its contents. We have also found that the language makes it easy to add features of a module as needed by adding the extra arguments and defining their uses as needed.

5.5 Formal Query Language Knowledge Base

As described briefly earlier, we make use of a Knowledge Base in compiling formal queries into the State Calculus. This Knowledge Base is essential in compiling high-level descriptions of packages, files, and so on from the Formal Query Language into the State Calculus. This is essential to the design of the Formal Query Language as it ensures that users do not accept incorrect but plausible sounding queries, which as we have described previously is an issue with LLM-based code generation. The knowledge base provides a set of functions that lower these high-level descriptions into lower-level ones appropriate for the State Calculus. For example, a `pkgDef` function takes in a package description and context information (including what OS is being targeted) and returns the appropriate package name and package manager; for instance, if the package description is “apache” or “apache server” and the OS being targeted is Debian-based it will return the package name `apache2` and the `apt` package manager while if the OS is RHEL it will return the name `httpd` and `dnf` package manager. Similarly, we have a `fileDef` function which given the description “zsh configuration file for user=foo” or “zshrc file for user=foo” will respond that this refers to the `.zshrc` file in foo’s home directory on the remote machine.

In our implementation, we define an interface for the knowledge base which contains these various functions we need and then parameterize our compiler by an instance of this interface. This makes it easy to replace and modify the knowledge base in the future. Currently, our knowledge base has been hand curated but it can be easily extended in the future and we are interested in exploring how to automate the construction by leveraging existing trustworthy knowledge sources, such as publicly available lists of packages and documentation such as Linux manual pages.

6 Evaluation

To evaluate our approach, we study Astrogator’s ability to differentiate correct and incorrect LLM-generated code. We present a benchmark suite of natural language descriptions and formal queries for common Ansible tasks (Section 6.1), describe our code-generation methodology and the quality of the LLM generated code (Section 6.2), and finally discuss the system’s accuracy (Section 6.3).

6.1 Benchmark Suite

For our evaluation, we need natural language descriptions and formal language queries for tasks to be done in Ansible. Prior work has used the name fields from tasks and plays as natural language descriptions [37] but in our experience these fields are generally quite short and often lack important details, like operating systems and paths.

Given the lack of a suitable existing benchmark suite, we develop our own by identifying common tasks based on top posts on StackOverflow and the Ansible Forum. In addition, we add tasks based on examples found in Ansible Galaxy, a large repository of Ansible code, some interesting variations on other tasks, and tasks that highlight challenges described earlier. For each benchmark we develop a natural language description, a formal language query, and a reference solution. We have produced a set of 21 benchmarks which require an average of 1.67 tasks in our reference solutions and a maximum of 4 tasks, putting them in line with the size of playbooks generated by prior work [37]. The natural language descriptions and formal queries included in our benchmark suite are provided in our Supplemental Materials; the examples in Table 1 are taken from the suite. The benchmark suite includes a mix of tasks involving file, user, and package operations.

Correctness checking requires a validated “ground-truth” program version we can use for comparisons. To do this we use tests because, even though they are imperfect, no other solution was feasible: as described in Section 6.2 we have 1260 programs, making manual inspection infeasible. We have done our best to craft strong tests that address subtle aspects of our problems. To test generated programs, we run them on a set of virtual machines along with setup and testing scripts which determine whether the program behaved as expected. We use multiple tests to test the behavior under different initial conditions, such as for problem 6 where a file should only be created if it does not already exist. We use VMs running Debian 12.11.0, Ubuntu 24.04.2, and Red Hat Enterprise Linux (RHEL) 9.6; for any problem that does not specify an operating system we test the program on each system and for problems that specify certain operating systems we run it on those systems. Between each program execution we reset the VMs from a snapshot to ensure that a program cannot impact the behavior of another. A program is judged to be correct only if it passes all tests on all systems it was tested on.

6.2 LLM Code Generation

To generate a diverse set of programs to test Astrogator, we have selected six LLMs and since the LLMs are nondeterministic we prompt each for ten programs for each problem to further increase the diversity of our program set, resulting in a total of 1,260 programs. The prompt we use is included in our Supplemental Materials; it gives basic instructions and an instruction not to make use of shell commands, such as the `shell` and `command` modules which allow the execution of arbitrary shell code, because we do not currently support these modules and their use is unnecessary in our benchmarks and therefore discouraged [39]. Despite this instruction we find 130 instances of generated code containing them; these programs are identified as incorrect by our verifier even though in some cases they do pass our tests.

In our experiments we use five open source-models which perform well on other code-generation benchmarks: Deepseek Coder 6.7b [20], Granite 8b Code [32], Qwen2.5 Coder 3b [22], Llama 3.1 8b [18], and Starcoder 2 15b [31]. We also use a state-of-the-art closed-source model GPT-4o [34]. We have observed that the models do not always generate full playbooks, in some cases models only generate a list of tasks; to address this, and ensure the playbooks work with our testing setup, we perform post-processing to identify and extract key elements and insert them into a template. In this process, we fail to process 42 (3.3%) of the generated programs due to syntactic issues. We also identified 5 programs (0.4%), all solutions to problem 21, which do not terminate because

Model	Error			
	S	E	T	C
Deepseek Coder 6.7b	0	87	70	53
GPT-4o	2	65	35	108
Granite-8b Code	0	89	66	55
Llama 3.1 8b	22	115	40	33
Qwen2.5 Coder	4	155	20	31
Starcode2 2 15b	14	90	52	54
Total	42	601	283	334

Table 2. Summary of the error types of code generated by model. Each model produced a total of 210 programs. The columns count the number of programs with **S**yntax Errors, **E**xecution Errors, **T**est Failures, and programs that are **C**orrect. The programs which do not terminate are counted as Execution Errors and programs manually identified to be incorrect code are counted as Test Failure errors.

#	Correct Programs		Incorrect Programs	
	Accepted	Rejected	Accepted	Rejected
1	55	0	5	0
2	56	0	0	4
3	5	2	0	53
4	0	0	49	11
5	5	4	0	51
6	11	0	4	45
7	3	0	0	57
8	2	0	0	58
9	37	7	0	16
10	5	22	0	33
11	42	0	2	16
12	11	0	3	46
13	0	0	0	60
14	0	6	0	54
15	0	0	0	60
16	14	8	0	38
17	24	0	2	34
18	0	1	2	57
19	7	0	3	50
20	0	7	0	53
21	0	0	0	60
Total	277	57	70	856

Table 3. Evaluation Results for the Benchmark Suite. For each benchmark we have 60 samples for a total of 1260 programs.

they invoke a shell command which requires user input. Of the remaining programs, we find that 596 (47.3%) fail to execute and 226 (18.0%) execute but fail the tests. In addition, the solutions to problem 18 cannot be executed (they attempt to access a URL that does not exist) but through manual inspection we identified only 1 of the 58 syntactically valid responses is correct. This means that in total, of the 1260 generated programs only 334 (26.5%) are correct.

We break down the performance of each model in Table 2. GPT-4o performs significantly better than the other models, generating 108 correct programs, or 51.4%, in contrast to only 21.5% from the open source models. This difference is most likely because the open source models are much smaller than GPT-4o. Within the open source models, Deepseek, Granite, and Starcoder all achieve slightly over 25% while Llama and Qwen only achieve around 15%. Qwen is the smallest model with only 3 billion parameters and Llama is the only non-code model included in our evaluation, which explains their low performance and may particularly explain Llama’s significantly higher than average rate of syntax errors.

6.3 Verifier Accuracy

To determine how well Astrogator works at differentiating correct from incorrect code, we run it on each on the programs generated by the LLMs using our hand-written formal queries. A perfect result would be that it identifies exactly the 334 correct programs as correct and all other programs as incorrect but some programs that are correct are rejected and some programs that are not correct are accepted. Our overall results, by benchmark program, are shown in Table 3.

Of the 334 correct programs Astrogator accepts 277 of them (82.9%) and of the 926 incorrect programs Astrogator rejects 856 of them (92.4%). In total, Astrogator took about 70 seconds to process the 1260 programs. In this process it detects many kinds of errors, including all of the errors

we have used as examples throughout this paper, as well as many errors involving using incorrect modules or module names, providing invalid arguments or missing necessary ones, providing invalid argument values, and so on. For the remainder of this section, we discuss the cases where Astrogator is incorrect and how these issues can be addressed.

Of the 57 correct programs that Astrogator rejects, 18 are because they rely on features we do not support yet; 9 of these rely on features like block constructs and handler, which are easy to support, while other 9 rely on shell commands which will be more difficult to support, as it requires a specification of the behavior of shell programs, we believe this is possible and discuss the possibility in Section 7.2. The other 39 programs come from problems: 9, 10, 16, and 20; these all share a common issue: the knowledge base is only allowed to produce one response and these programs used a correct value (or approach) that did not match the one the knowledge base provided. For example, in problem 9 we cloning a git repository and the programs use the address `https://github.com/ansible/ansible` while knowledge base produced this same address with `.git` at the end. Similarly, problem 10 which sets an environment variable, expects that we insert the line `LC_ALL=C` while the line `LC_ALL="C"` works as well. Problem 16's issue is the same, the knowledge base provides the path `/bin/zsh` for `zsh` but the path `/usr/bin/zsh` work as well. Again, for problem 20 where we enable `sudo` for a group, the knowledge base provides a particular line to be added to the `/etc/sudoers` file but multiple acceptable lines exist and it can be added to other files as well. The simple solution to these is to allow the knowledge base to return multiple values, which would fix these issues, but unfortunately the implementation is not quite as trivial, while we believe it can be resolved it is likely to be rather technical and may require changes to our handling of symbolic values. Thus, all of these incorrect rejections are likely fixable, though require varying levels of effort to fix.

Of the 70 incorrect programs that Astrogator accepts we find that 68 are the result of the queries being under-specified and that we fact that we are accepting all programs that satisfy the query (i.e., without a human verifying $p \setminus s$). of the other two programs, one is the result of an unhandled feature, Shell plugins, which in compiling the Ansible to the State Calculus we simply ignore but causes Ansible to fail to execute it; handling Shell plugins in our compilation of Ansible would immediately address this issue. The other program is due to an undocumented feature in the `lineinfile` module [2], this issue can be easily fixed by adjusting the module's definition given in the module description language¹. Of the other 68 programs, 7 perform undesired additional actions that caused our tests to fail while the remaining 61 would work under certain assumptions not true in our testing environment; these issues are not unexpected given that as we have described formal queries are generally under-specified, these issues are entirely expected and as discussed in Section 3 they are resolved by asking a user for input. These 7 aforementioned programs are for problems 6 (creating a file if it does not exist) and 12 (rebooting the system if needed), which both involve conditionals like "if *cond* then do *foo*". Intuitively it seems such a query should also mean "... otherwise do not do *foo*" but since programs must be allowed to perform additional actions this is not the semantics of the formal query language.

Finally, the other 61 incorrect programs accepted by Astrogator work under certain additional assumptions, which again is expected. As an example, the five incorrect programs for problem 1 fail because they set the owner of the newly created directory to a user that did not exist. The output of the unifier in these cases reports that they assume the user exists and they own the new directory, and a user could easily identify whether this matches their intent. There is a similar issue with problem 17 where the two incorrect programs work if the "service" user does not already

¹We did not fix this simply because it is not clear if we should model the documented behavior (i.e., the actual behavior is a bug), the actual behavior (i.e., the behavior is intentional), or simply reject any program that triggers this behavior

exist which a user can easily check. The remaining issues involve the creation of files with certain contents, for example in problem 4 we are meant to copy a file on the remote machine while the generated code copies the file from the controller to the remote which is not rejected because the target files are ultimately created but it requires the alternate file sources to have certain contents; again these assumptions are identified by the unifier and a user could easily review them.

As discussed above, while Astrogator is not always correct on whether a program is correct or not, its incorrect rejections are all addressable by adding support for additional features and the incorrect accepts are all expected as they are the result of the under-specified nature of the formal queries, the solution for them is to include a user to check whether the additional actions or assumptions they require match their intent. These results prove that the techniques and ideas presented work and the failures are due to current limitations in our implementation.

7 Discussion

7.1 Engineering Effort

Astrogator has taken a significant amount of effort to develop and this could be a limiting factor to the adoption of our proposed approach. Astrogator was developed by one author in less than a year and while Astrogator is still a research project and not fully featured, we believe that most of the additions to make it more full featured are relatively simple, such as writing specifications in the module language and handling the compilation of additional Ansible features. Moreover, this investment of time and resources pales in comparison to the amount of time and resources required for the development of a LLM for code generation.

We also emphasize that this work is only a first step and it has shown the strong potential of our approach. We aim to reduce the necessary engineering effort in future work. In particular, we believe there are opportunities to automate the construction of many components such as the knowledge base, parts of the formal query language, and components like module specification; one approach we are considering is to provide LLMs with documentation and other reference materials, asking them to generate the needed components, and then reviewing the results manually.

7.2 Generality

While Astrogator is specific to Ansible the ideas presented are relevant to other languages and the State Calculus and its symbolic interpreter may be useful as part of verifying other stateful languages. In particular, we can already support a subset of bash programs and microcontroller programs, both of which are critical to correctness of many systems. We use two examples to illustrate these capabilities, and leave it to future work to expand the support for each.

Bash Scripts. Figure 6 shows a Bash script, a hand-translation of that script into the State Calculus, and a hand written specification in the State Calculus that could easily be derived from a formal query language for Bash. Using these, and definitions of the Linux utilities used in this script, we have been able to use Astrogator to verify that the hand-translation of the script matches the hand-written specification. This example demonstrates a number of Bash features which are quite distinct from Ansible, including redirects, pipes, and even file descriptors. One feature that we anticipate may be challenging to support are arbitrary loops; as described in Section 5.2.2 our handling of loops over symbolic values is pretty simple and does not support arbitrary while-loops or more complex forms of loops like found in languages like Bash. We believe that even without this support, a significant and important subset of Bash can be supported; expanding to support these loops may be quite difficult as arbitrary loops are a challenge common challenge in verification.

883			1	<code>let</code> pipe = new_pipe();
884	<code>tee >(wc -l >&2) < bigfile gzip > bigfile.gz</code>		2	<code>let</code> file = tmp_file();
885			3	<code>fd(0).kind = fd::file('bigfile');</code>
886	(a) Example Bash script		4	<code>fd(1).kind = fd::pipe(pipe);</code>
887	1 <code>let</code> contents = fs('bigfile').contents;		5	<code>tee {{ files: [file] }}</code> };
888	2		6	<code>fd(0).kind = fd::file(file);</code>
889	3 <code>env().stderr = ((env().stderr</code>		7	<code>fd(1).kind = fd(2).kind;</code>
890	4 <code>^ string_of_int(count_lines(contents))</code>		8	<code>wc {{ lines: true }}</code> };
891	5 <code>^ " " ^ "\n";</code>		9	<code>fd(0).kind = fd::pipe(pipe);</code>
892	6 <code>fs('bigfile.gz').contents = gzipped(contents);</code>		10	<code>fd(1).kind = fd::file('bigfile.gz');</code>
893			11	<code>gzip {{ }}</code> };
894				
895	(b) Hand-written specification			(c) Hand-translation of Bash script

Fig. 6. Example translation of a Bash script to the State Calculus. The script prints the number of lines in bigfile to stderr and gzips it to bigfile.gz.

897	1 <code>const int</code> BUTTON = 2;		1	<code>let</code> time = any_time();
898	2 <code>const int</code> LED = 8;		2	<code>let</code> oldState = globals().buttonState;
899	3 <code>int</code> buttonState;		3	<code>globals().buttonState = digitalRead(2);</code>
900	4 <code>int</code> ledState;		4	<code>if</code> !oldState && globals().buttonState {
901	5 <code>void</code> setup() {		5	<code>globals().ledState = ! globals().ledState;</code>
902	6 <code>pinMode</code> (LED, OUTPUT);		6	<code>digitalWrite(8, globals().ledState);</code>
903	7 <code>pinMode</code> (BUTTON, INPUT);		7	}
904	8 <code>ledState = LOW;</code>		8	
905	9 <code>digitalWrite</code> (LED, ledState);			
906	10 <code>buttonState = digitalRead</code> (BUTTON);			
907	11 }			
908	12 <code>void</code> loop() {			(b) Hand-translation of Arduino program
909	13 <code>int</code> oldState = buttonState;		1	<code>let</code> cur_time = any_time();
910	14 <code>buttonState = digitalRead</code> (BUTTON);		2	<code>let</code> prev_time = cur_time - 1;
911	15		3	<code>if</code> !input(prev_time).pin(2).pinValue
912	16 <code>if</code> (oldState==LOW&&buttonState==HIGH) {		4	&& input(cur_time).pin(2).pinValue {
913	17 <code>ledState = !ledState;</code>		5	<code>output</code> (cur_time).pin(8).pinValue =
914	18 <code>digitalWrite</code> (LED, ledState);		6	! output(prev_time).pin(8).pinValue;
915	19 }		7	}
916	20 }			
917				
918	(a) Example Arduino program			(c) Hand-written specification

Fig. 7. Example translation of an Arduino program to the State Calculus. The program toggles an LED each time a button is clicked.

Arduino Programs. Figure 7 shows an Arduino² program along with a hand-translation into the State Calculus and a hand-written specification that we are able to verify matches the hand-translation. In this example we translate the main loop of the program which acts as a state machine and we simply verify that it acts correctly at each step. In this setting, verification using Astrogator may need to be paired with other verification techniques to ensure, for example, that expected invariants (such as assumptions about globals that arise from the State Calculus's unifier) are satisfied.

We highlight that in both examples, like with Ansible, the translation from the source language to the Calculus is straightforward, the main source of complexity comes from the language specification, for Bash this includes the utilities and for Arduino the built-in functions. While both of these examples rely on hand-translation of the source programs and there is still significant

²Arduino is an open-source microcontroller, it is programmed in a C-like language [5].

work to be done on determining the best way to represent the state for these languages, these examples strongly suggest to us that the State Calculus can be used to verify the behavior of Bash and Arduino programs and likely programs in other similar languages as well.

8 Related Work

Controlled Natural Languages. There has been significant research on Controlled Natural Languages (CNLs) [17, 21, 26, 41] which have a syntax based on a restricted form of natural language. CNLs have been used in various applications, from database query languages [21] to first-order logic specifications [17, 41]. The formal query languages we propose in this work are similar to CNLs, which bolsters our belief that formal queries can be easily understood by users. To our knowledge, CNLs have not been used before to formalize user intent for LLM code-generation.

Identifying Incorrect LLM Generated Code. Some prior work has used test cases to check LLM generated code [11], which increases the likelihood that the generated code is correct and with a sufficient test suite could offer strong assurances of correctness but this requires the user to craft good tests; which is so hard that errors were found in the ground-truth solutions to the well known HUMAN-EVAL benchmark suite [29] despite their tests.

Verified LLM Generated Code. LLM code generation has been explored for generating proofs in proof assistants such as Rocq and Isabelle [16] because the LLM generated proof can be checked for correctness by the theorem prover, but this approach does not generalize to general code where oracles for verifying correctness do not exist. This work also relies on a user specified formal theorem statement, though auto-formalization has been used to convert natural language theorem statements into formal ones [42], but the generated theorem statements are in a lower-level language than our formal query language making them more difficult for users to check.

Natural Language Summaries. Showing the user a natural language-like description to check has been proposed before but prior work has explored providing a description of a generated program [8, 30]. While this can help a user determine if the generated code matches their intent we believe our approach of having the user verify a high-level specification is better since it helps avoid issues where a code summary may seem reasonable but contain subtle errors.

LLM Generation of Formal Specifications. The use of LLMs to formalize a user’s intent has been explored in other contexts such as generating post-conditions [15] and generating Isabelle theorem statements [42]. These works show that LLMs can generate reasonable formalizations which can then be used to check generated code or proofs. However, we believe that the formalizations chosen by these prior works are not suited for the user to check correctness of as these are significantly lower-level than the the natural language description.

Program Synthesis. Finally, our work is an attempt to merge LLM-based code generation with ideas and techniques from Program Synthesis. In particular, in our vision for a complete code-generation system (Figure 2) the loop between the Code Generator and Verifier bears significant resemblance to the feedback loop found in Counterexample Guided Inductive Synthesis, and similar techniques. Additionally, the user’s input in the Assumptions Check phase bears resemblance to interactive synthesis [35] where the user may iteratively refine the specification until the generated program matches their full intent. There has also been some prior work that makes use of both LLM code generation and program synthesis, such as Photoscout [8] which uses an LLM to generate programs and then a semantic analysis identifies undefined components and replaces them with holes which are filled using an enumerative synthesis approach. This technique could be relevant to Ansible, as our formal queries could provide a basis to perform such synthesis.

9 Conclusion

In this work we have proposed the use of *formal query languages*, which formalize a user’s intent, to check LLM generated code for correctness. We present design constraints for such languages, to ensure a user can check that a query matches their intent, and present our implementation, Astrogator, for the Ansible IT automation language and show that we can use our verifier to effectively identify correct and incorrect LLM generated code. While the verifier is imperfect, we analyze the errors and find that the errors are all results of limitations in our testing setup or in the implementation that could be easily addressed. We believe this work can lay groundwork for building systems which, from a natural language description, are able to generate code that is guaranteed to match the user’s intent, which is essential to allow AI Code Assistants to be adopted in critical systems.

References

- [1] Mouhamad Almakhour, Layth Sliman, Abed Ellatif Samhat, and Abdelhamid Mellouk. 2020. Verification of smart contracts: A survey. *Pervasive and Mobile Computing* 67 (2020), 101227. doi:10.1016/j.pmcj.2020.101227
- [2] Ansible Project Contributors. 2025. ansible.builtin.lineinfile module – Manage lines in text files. https://docs.ansible.com/ansible/latest/collections/ansible/builtin/lineinfile_module.html
- [3] Ansible Project Contributors. 2025. Creating a playbook. https://docs.ansible.com/ansible/latest/getting_started/get_started_playbook.html
- [4] Ansible Project Contributors. 2025. Getting started with Ansible. https://docs.ansible.com/ansible/latest/getting_started/index.html
- [5] Arduino. 2025. Language Reference. <https://docs.arduino.cc/language-reference>
- [6] Tom Badgett, Glenford J. Myers, and Corey Sandler. 2011. *The Art of Software Testing, 3rd Edition*. Wiley. doi:10.1002/9781119202486
- [7] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 78 (April 2023), 27 pages. doi:10.1145/3586030
- [8] Celeste Barnaby, Qiaochu Chen, Chenglong Wang, and Isil Dillig. 2024. PhotoScout: Synthesis-Powered Multi-Modal Image Search. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI ’24). Association for Computing Machinery, New York, NY, USA, Article 896, 15 pages. doi:10.1145/3613904.3642319
- [9] Samera Obaid Barraood, Haslina Mohd, and Fauziah Baharom. 2021. Test Case Quality Factors. *Turkish Journal of Computer and Mathematics Education* 12, 3 (2021), 1683–1694.
- [10] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT: Code Generation with Generated Tests. In *The Eleventh International Conference on Learning Representations*. 19 pages. <https://openreview.net/forum?id=ktrw68Cmu9c>
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [12] Edsger W. Dijkstra. 1979. On the foolishness of “natural language programming”. In *Program Construction: International Summer School*, Friedrich L. Bauer, Manfred Broy, E. W. Dijkstra, S. L. Gerhart, D. Gries, M. Griffiths, J. V. Guttag, J. J. Horning, S. S. Owicki, C. Pair, H. Partsch, P. Pepper, M. Wirsing, and H. Wössner (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 51–53. doi:10.1007/BFb0014656
- [13] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Weikang Zhou, Muling Wu, Mingxu Chai, Jessica Fan, Caishuang Huang, Yunbo Tao, Yan Liu, Enyu Zhou, Ming Zhang, Yuhao Zhou, Yueming Wu, Rui Zheng, Ming Wen, Rongxiang Weng, Jingang Wang, Xunliang Cai, Tao Gui, Xipeng Qiu, Qi Zhang, and Xuanjing Huang. 2024. What’s Wrong with Your Code Generated by Large Language Models? An Extensive Study. arXiv:2407.06153 [cs.SE] <https://arxiv.org/abs/2407.06153>

- [14] Sigrid Eldh, Hans Hansson, and Sasikumar Punnekkat. 2011. Analysis of Mistakes as a Method to Improve Test Case Design. In *2011 Fourth IEEE International Conference on Software Testing, Verification and Validation*. 70–79. doi:10.1109/ICST.2011.52
- [15] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions? *Proc. ACM Softw. Eng.* 1, FSE, Article 84 (July 2024), 24 pages. doi:10.1145/3660791
- [16] Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. 2023. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (San Francisco, CA, USA) (ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1229–1241. doi:10.1145/3611643.3616243
- [17] Robert E. Fuchs, Uta Schwertel, and Sunna Torge. 1999. Controlled Natural Language Can Replace First-Order Logic. In *14th IEEE International Conference on Automated Software Engineering*. 295–298. doi:10.1109/ASE.1999.802325
- [18] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmervan der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Young, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoqiang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Conguet, Virginie Do, Vish Vogeti, Vitor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank

- Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Sweet, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damla, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabza, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michael Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojuan Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783>
- [19] Xiaodong Gu, Meng Chen, Yalan Lin, Yuhang Hu, Hongyu Zhang, Chengcheng Wan, Zhao Wei, Yong Xu, and Juhong Wang. 2025. On the Effectiveness of Large Language Models in Domain-Specific Code Generation. *ACM Transactions on Software Engineering and Methodology* 34, 3 (Feb. 2025), 1–22. doi:10.1145/3697012
- [20] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196 [cs.SE] <https://arxiv.org/abs/2401.14196>
- [21] Catalina Hallett, Donia Scott, and Richard Power. 2007. Composing Questions through Conceptual Authoring. *Computational Linguistics* 33, 1 (03 2007), 105–133. doi:10.1162/coli.2007.33.1.105
- [22] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL] <https://arxiv.org/abs/2409.12186>
- [23] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. 51 pages. <https://openreview.net/forum?id=VTF8yNQm66>
- [24] Satvik Joel, Jie JW Wu, and Fatemeh H. Fard. 2024. A Survey on LLM-based Code Generation for Low-Resource and Domain-Specific Programming Languages. arXiv:2410.03981 [cs.SE] <https://arxiv.org/abs/2410.03981>
- [25] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: Formal Verification of an OS Kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) (SOSP '09). Association for Computing Machinery, New York, NY, USA, 207–220. doi:10.1145/1629575.1629596
- [26] Tobias Kuhn. 2014. A Survey and Classification of Controlled Natural Languages. *Computational Linguistics* 40, 1 (03 2014), 121–170. doi:10.1162/COLI_a_00168
- [27] Xavier Leroy. 2006. Formal Certification of a Compiler Back-end or: Programming a Compiler with a Proof Assistant. In *Conference Record of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Charleston,

- South Carolina, USA) (*POPL '06*). Association for Computing Machinery, New York, NY, USA, 42–54. doi:10.1145/1111037.1111042
- [28] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (*ICSE '24*). Association for Computing Machinery, New York, NY, USA, Article 52, 13 pages. doi:10.1145/3597503.3608128
- [29] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 21558–21572. https://proceedings.neurips.cc/paper_files/paper/2023/file/43e9d647ccd3e4b7b5baab53f0368686-Paper-Conference.pdf
- [30] Michael Xieyang Liu, Advait Sarkar, Carina Negreanu, Benjamin Zorn, Jack Williams, Neil Toronto, and Andrew D. Gordon. 2023. “What It Wants Me To Say”: Bridging the Abstraction Gap Between End-User Programmers and Code-Generating Large Language Models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (*CHI '23*). Association for Computing Machinery, New York, NY, USA, Article 598, 31 pages. doi:10.1145/3544548.3580817
- [31] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2024. StarCoder 2 and The Stack v2: The Next Generation. arXiv:2402.19173 [cs.SE] <https://arxiv.org/abs/2402.19173>
- [32] Mayank Mishra, Matt Stallone, Gaoyuan Zhang, Yikang Shen, Aditya Prasad, Adriana Meza Soria, Michele Merler, Parameswaran Selvam, Saptha Surendran, Shivdeep Singh, Manish Sethi, Xuan-Hong Dang, Pengyuan Li, Kun-Lung Wu, Syed Zawad, Andrew Coleman, Matthew White, Mark Lewis, Raju Pavuluri, Yan Koyfman, Boris Lublinsky, Maximilien de Baysse, Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal, Yi Zhou, Chris Johnson, Aanchal Goyal, Hima Patel, Yousaf Shah, Petros Zefos, Heiko Ludwig, Asim Munawar, Maxwell Crouse, Pavan Kapanipathi, Shweta Salaria, Bob Calio, Sophia Wen, Seetharami Seelam, Brian Belgodere, Carlos Fonseca, Amith Singhee, Nirmal Desai, David D. Cox, Ruchir Puri, and Rameswar Panda. 2024. Granite Code Models: A Family of Open Foundation Models for Code Intelligence. arXiv:2405.04324 [cs.AI] <https://arxiv.org/abs/2405.04324>
- [33] Dung Manh Nguyen, Thang Chau Phan, Nam Le Hai, Tien-Thong Doan, Nam V. Nguyen, Quang Pham, and Nghi D. Q. Bui. 2025. CodeMMLU: A Multi-Task Benchmark for Assessing Code Understanding & Reasoning Capabilities of CodeLLMs. In *The Thirteenth International Conference on Learning Representations*. 59 pages. <https://openreview.net/forum?id=CahIEKCUsQ>
- [34] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Lukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Lukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv

- Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichi Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vithyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] <https://arxiv.org/abs/2303.08774>
- [35] Hila Peleg, Shachar Itzhaky, and Sharon Shoham. 2018. Abstraction-Based Interaction Model for Synthesis. In *Verification, Model Checking, and Abstract Interpretation*, Isil Dillig and Jens Palsberg (Eds.). Springer International Publishing, Cham, 382–405. doi:10.1007/978-3-319-73721-8_18
- [36] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) (CCS ’23). Association for Computing Machinery, New York, NY, USA, 2785–2799. doi:10.1145/3576915.3623157
- [37] Saurabh Pujar, Luca Buratti, Xiaojie Guo, Nicolas Dupuis, Burn Lewis, Sahil Suneja, Atin Sood, Ganesh Nalawade, Matthew Jones, Alessandro Morari, and Ruchir Puri. 2023. Automated Code Generation for Information Technology Tasks in YAML through Large Language Models. arXiv:2305.02783 [cs.SE] <https://arxiv.org/abs/2305.02783>
- [38] Red Hat. 2025. What is Ansible? <https://www.ansible.com>
- [39] Red Hat, Inc. 2025. command-instead-of-module, Ansible Lint Documentation. <https://ansible.readthedocs.io/projects/lint/rules/command-instead-of-module/#correct-code>
- [40] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] <https://arxiv.org/abs/2308.12950>
- [41] Rolf Schwiter. 2002. English as a Formal Specification Language. In *Proceedings. 13th International Workshop on Database and Expert Systems Applications*. 228–232. doi:10.1109/DEXA.2002.1045903
- [42] Yuhuai Wu, Albert Qiaoju Jiang, Wenda Li, Markus Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with Large Language Models. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 32353–32368. https://proceedings.neurips.cc/paper_files/paper/2022/file/d0c6bc641a56bebee9d985b937307367-Paper-Conference.pdf
- [43] Daniel M. Yellin. 2023. The Premature Obituary of Programming. *Commun. ACM* 66, 2 (Jan. 2023), 41–44. doi:10.1145/3555367
- [44] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA022 (June 2025), 23 pages. doi:10.1145/3728894
- [45] Terry Yue Zhuo, Vu Minh Chien, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen GONG, James Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, David Lo, Binyuan Hui, Niklas Muennighoff, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *The Thirteenth International Conference on Learning Representations*. 55 pages. <https://openreview.net/forum?id=YrcTjllL0>
- [46] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN*

1226

1227

1228

1229

1230

1231

1232

1233

1234

1235

1236

1237

1238

1239

1240

1241

1242

1243

1244

1245

1246

1247

1248

1249

1250

1251

1252

1253

1254

1255

1256

1257

1258

1259

1260

1261

1262

1263

1264

1265

1266

1267

1268

1269

1270

1271

1272

1273

1274

International Symposium on Machine Programming (San Diego, CA, USA) (*MAPS 2022*). Association for Computing Machinery, New York, NY, USA, 21–29. doi:[10.1145/3520312.3534864](https://doi.org/10.1145/3520312.3534864)